

Ch. 14

命名空間、引用與組件

(Namespace, using & DLL)

Horazon

C#程式設計

為什麼需要命名空間？

假設你和同學都寫了一個叫做 `Player` 的類別。

兩個 `Player` 放在同一個程式裡，編譯器會不知道要用哪一個！

命名空間 (Namespace) 就像「姓氏」，幫類別加上識別標誌：

```
// 你寫的  
MyGame.Player  
  
// 同學寫的  
HisGame.Player
```

這樣兩個 `Player` 就不會衝突了。

宣告命名空間

用 `namespace` 關鍵字包住你的類別：

```
namespace MyGame
{
    class Player
    {
        public string Name = "勇者";
    }
}
```

在另一個地方使用時，需要寫完整名稱：

```
MyGame.Player hero = new MyGame.Player();
```

using 指令

每次都寫完整名稱很麻煩。

使用 `using` 可以告訴編譯器「我要用這個命名空間」：

```
using MyGame; // 引入命名空間  
  
// 現在可以直接用，不需要寫 MyGame.  
Player hero = new Player();
```

這就是為什麼每支程式開頭都會看到：

```
using System;  
using System.Collections.Generic;
```

巢狀命名空間

命名空間可以有層級，用 `.` 分隔：

```
namespace MyGame.Enemies
{
    class Slime { }
    class Goblin { }
}

namespace MyGame.Items
{
    class Potion { }
}
```

使用時：

```
using MyGame.Enemies;

Slime s = new Slime();
```

.NET 常見命名空間

命名空間	內容
<code>System</code>	基本型別、Console、Math
<code>System.Collections.Generic</code>	List、Dictionary
<code>System.IO</code>	檔案讀寫
<code>System.Text</code>	字串處理 (StringBuilder)
<code>System.Linq</code>	資料查詢
<code>UnityEngine</code>	Unity 核心功能

什麼是 DLL？

DLL (Dynamic Link Library) 是已經編譯好的程式碼，打包成一個 `.dll` 檔案。

- 你寫的 C# 程式碼 `.cs` → 編譯後 → `.dll`
- 別人寫好的功能，直接引用 `.dll` 就能使用，不需要原始碼。

生活比喻

就像買現成的「零件」組裝：

不需要自己製造馬達，直接買來裝上就好。

引用 DLL (組件)

在 Visual Studio 中，可以手動加入 DLL 參考：

1. 在專案上右鍵 → **新增參考** (Add Reference)
2. 選擇 `.dll` 檔案
3. 在程式碼加上對應的 `using`

```
// 引用後，才能使用裡面的命名空間
using SomeLibrary;

var obj = new SomeLibrary.MyClass();
```


全域 using (C# 10+)

在 C# 10 之後，可以在專案層級定義「全域 using」，讓整個專案所有檔案都自動引入，不需要每個檔案重複寫：

```
// GlobalUsings.cs  
global using System;  
global using System.Collections.Generic;  
global using UnityEngine;
```

現代 Unity 和 ASP.NET Core 專案都大量使用這個功能。

最上層陳述式 (Top-level Statements)

傳統 C# 程式需要大量「儀式性」的樣板程式碼：

```
using System;

namespace MyApp
{
    class Program
    {
        static void Main(string[] args)
        {
            Console.WriteLine("Hello World");
        }
    }
}
```

對初學者來說，真正要學的只有第 9 行，其他都是「規定要寫的」。

進入點：Main 方法

每個程式都需要一個「從哪裡開始執行」的地方，這叫做 **進入點 (Entry Point)**。

在 C# 中，進入點固定是名為 `Main` 的靜態方法：

```
static void Main(string[] args)
{
    // 程式從這裡開始執行
}
```

- 程式啟動時，作業系統會**直接呼叫** `Main`。
- `Main` 執行完畢後，程式結束。
- `args` 是執行程式時可以從外部傳入的參數。

最上層陳述式 (C# 9+)

C# 9 之後，可以省略 `class Program` 和 `Main`，直接寫邏輯，甚至連using都消失了：

```
// 這就是完整的程式！
```

```
Console.WriteLine("Hello World");
```

- 編譯器會**自動**幫你產生 `Main` 方法。
- 原本需要寫 `using System;`
- **整個專案只能有一個檔案**使用最上層陳述式。

概念	說明
namespace	幫類別加上「姓氏」，避免命名衝突
using	省略命名空間前綴，讓程式碼更簡潔
DLL	編譯後的程式庫，可直接引用使用
global using	C# 10+ 的全域引用，減少重複 using